

Я начал делать этап:

«ЭТАП 20: Подготовка Example Frontend (Bonus)

Цель: Реализовать бонус "Example Frontend — Simple UI using the TypeScript SDK". Это докажет, что наш SDK работает не только в консоли (Node.js), но и в браузере (React).

Шаги:

1. **Инициализация:** В папке `examples/frontend/` создать Next.js (или Vite) проект.
2. **Интеграция:** Установить `@solana/wallet-adapter-react` и наш локальный пакет `@stbr/sss-token` (наш SDK из папки `sdk/`).
3. **UI:** Сделать очень простую страницу "Bank Portal".
 - Кнопка "Connect Wallet".
 - Карточка с балансом (Total Supply стейблкоина).
 - Две формы: "Mint Tokens" и "Add to Blacklist".
4. **Логика:** При нажатии "Mint" вызывается метод из нашего SDK, кошелек (Phantom) просит подпись администратора (если он подключен), транзакция уходит.
5. **Важно:** В этом фронтенде **не нужно** делать сложный дизайн (как мы делали в LMS). В ТЗ сказано "Simple UI". Судьи будут смотреть именно на чистоту кода вызова функций нашего SDK.

»

Но выходит ошибка при подписании транзакции в phantom:

«

App.tsx:108 WalletSendTransactionError: Unexpected error at async handleMint (App.tsx:108:31)

1. `StandardWalletAdapter {_events: Events, _eventsCount: 4, signIn: f, signTransaction: f, signAllTransactions: f, ...}`
- 2.

App.tsx:122 WalletSendTransactionError: Unexpected error at async handleMint (App.tsx:108:31)

»

Также нужно сделать следующие этапы:

«ЭТАП 21: Unit & Integration Тесты (TypeScript/Mocha)

Цель: Написать базовый набор тестов для смарт-контракта, который проверяет все позитивные и негативные сценарии (Happy Path & Sad Path).

Шаги:

1. **Настройка:** В папке `tests/` инициализировать Mocha/Chai (стандарт для Anchor).
2. **Тестирование SSS-1 (Minimal Stablecoin):**
 - Создать файл `sss-1-core.ts`.

- *Test 1:* Инициализация контракта (проверка, что Config PDA создан верно).
- *Test 2:* Успешный Mint от имени minter_authority.
- *Test 3 (Negative):* Попытка сделать Mint с кошелька, у которого нет роли minter_authority (ожидаем падение транзакции с кастомной ошибкой Unauthorized).
- *Test 4:* Проверка работы заморозки аккаунта (Freeze/Thaw).

3. Тестирование SSS-2 (Compliance):

- Создать файл sss-2-compliance.ts.
- *Test 1:* Добавление адреса в Blacklist.
- *Test 2: Ключевой тест Transfer Hook.* Попытка перевести токены с чистого адреса на адрес из Blacklist. (Ожидаем, что Transfer Hook прервет перевод).
- *Test 3:* Изъятие (Seize) токенов у замороженного пользователя через Permanent Delegate.

ЭТАП 22: Fuzzing Тестирование (Trident) — Wow Feature

Цель: Показать судьям высший пилотаж в безопасности (Fuzzing — это автоматическая подача случайных/мусорных данных в контракт для поиска уязвимостей). В ТЗ прямо сказано: "Fuzz via Trident".

Шаги:

1. **Установка Trident:** (Это фреймворк для Rust-фаззинга контрактов Solana). Установить CLI Trident (cargo install trident-cli).
2. **Инициализация:** В корне проекта запустить trident init. Это создаст папку trident-tests/.
3. **Написание Fuzz-теста:**
 - Написать фазз-тест для инструкции mint и burn.
 - *Смысл теста:* Передавать случайные огромные числа (например, u64::MAX) в поле amount, случайные ключи вместо minter_authority.
 - *Инвариант (Invariants):* Проверять, что после любых хаотичных действий Total Supply стейблкоина всегда равен сумме балансов всех кошельков (защита от печати денег из воздуха).
4. **Запуск:** Убедиться, что trident fuzz run отрабатывает без паники (panic) программы. Описать этот процесс в TESTING.md.

ЭТАП 23: Тестирование SDK (Local API Testing)

Цель: Убедиться, что наш TypeScript SDK корректно формирует запросы и возвращает понятные ошибки.

Шаги:

1. **Создание тестов для SDK:** В папке sdk/tests/ (или sdk/src/__tests__/) создать тесты с использованием Jest или Vitest.

2. **Мок-интерфейсы:** Так как тестировать блокчейн долго, написать тесты, которые просто проверяют *структуру* формируемых транзакций.
 - Например, тест проверяет: если мы вызываем `stable.create({ preset: Presets.SSS_2 })`, то в генерируемой транзакции точно есть инструкция инициализации Transfer Hook'a.
3. **Парсинг ошибок:** Проверить, что если контракт возвращает код ошибки 0x1771 (например, Wallet Blacklisted), то SDK перехватывает это и кидает красивую JS-ошибку `new BlacklistedError("This wallet is restricted")`, а не сырой лог блокчейна.

ЭТАП 24: Security Audit & Аннотации (Code Review)

Цель: Подготовить код к ручной проверке судьями, закрыть типичные уязвимости.

Шаги:

1. **Аудит Access Control:** Проверить все файлы в `programs/sss_core/src/instructions/`.
 - Убедиться, что **везде** используются PDA с правильными сидами.
 - Убедиться, что используется `#[access_control(check_role(...))]` перед телом функций, меняющих стейт.
2. **Защита от Replay Attacks:**
 - Так как у нас нет nonce в типичных инструкциях SPL (это делает сама Solana), убедиться, что мы не используем подписи сообщений (Signatures) вне стандартного механизма Solana. Если в SSS-3 (Privacy) используется кастомная криптография, внедрить проверку Nonce (как в примере `shadowpay: { nonce }` из прошлой задачи, если мы будем использовать подобный подход).
3. **Безопасная математика:** В Rust коде заменить все обычные плюсы и минусы (+, -) на `checked_add` и `checked_sub`.
 - *Пример:* Вместо `supply += amount` писать `supply = supply.checked_add(amount).unwrap()`. Это защита от переполнения (Overflow/Underflow).

ЭТАП 25: Stress-test на Devnet (Real-world Simulation)

Цель: Подтвердить, что архитектура не ломается при реальных сетевых задержках и взаимодействии программ.

Шаги:

1. **Скрипт нагрузки:** Написать Node.js скрипт `scripts/stress_test.ts`.
2. **Сценарий:** Скрипт должен:
 - Создать стейблкоин SSS-2 в Devnet.
 - Сгенерировать 50 случайных кошельков.
 - Сделать Mint на все 50 кошельков (проверка параллельных транзакций).

- Добавить 10 кошельков в Blacklist.
 - Попытаться сделать переводы между всеми 50 кошельками в цикле.
3. **Отчет:** Скрипт должен вывести в консоль: "Успешных переводов: X, Заблокировано Хуком: Y". Если все совпадает с ожидаемой математикой — тест пройден.

ЭТАП 26: Создание базовой документации (README & SSS Specs)

Цель: Создать лицо проекта. Файлы должны лежать в корне проекта и в папке docs/.

Шаги:

1. **README.md (в корне):**
 - **Заголовок:** "Solana Stablecoin Standard (SSS)".
 - **Badges:** Добавить бейджи (Build passing, License MIT, Version).
 - **Overview:** Краткое описание (3 слоя: SDK, Modules, Presets). Упомянуть реализацию бонусных фиш (SSS-3, TUI).
 - **Quick Start:** Как запустить проект в 3 команды (npm install, npm run build, npm run start).
2. **docs/SSS-1.md и docs/SSS-2.md:**
 - Спецификация стандартов. Расписать, какие расширения Token-2022 используются в каждом пресете (Minimal vs Compliant).
 - Добавить фрагменты кода из нашего SDK для создания каждого пресета.
3. **docs/OPERATIONS.md (Runbook):**
 - Написать инструкцию для оператора: как пользоваться нашим CLI/TUI.
 - Как заминтить токены, как добавить в Blacklist. Вставить скриншоты нашего TUI.

ЭТАП 27: Архитектурная документация (Mermaid & ASCII)

Цель: Показать глубокое понимание системы через визуализацию (требование из ТЗ).

Шаги:

1. **docs/ARCHITECTURE.md:**
 - Описать три слоя архитектуры (Base SDK, Modules, Presets).
 - **Mermaid-диаграмма:** Вставить код для рендеринга диаграммы потока (Data Flow) для SSS-2 (попытка перевода -> вызов Transfer Hook -> проверка Blacklist PDA -> возврат результата).
 - *Пример Mermaid:*

codeMermaid

sequenceDiagram

participant User

participant Token2022

```

participant TransferHook
User->>Token2022: Transfer 100 SSS
Token2022->>TransferHook: Execute Hook
TransferHook-->>TransferHook: Check Blacklist PDA
alt is Blacklisted
    TransferHook--xToken2022: Error: Blacklisted
else is Clean
    TransferHook-->>Token2022: Ok
    Token2022-->>User: Transfer Complete
end

```

- **ASCII-схема:** Нарисовать схематично структуру PDA (Program Derived Addresses) для ролей, как это просят организаторы. (Например: [Config PDA] --> [Minter PDA 1], [Minter PDA 2]).

2. docs/SECURITY.md:

- Описать векторы атак и как они закрыты.
Упомянуть #[access_control], Fuzzing-тесты Trident (Этап 22).

ЭТАП 28: Докеризация (Docker Compose)

Цель: Обеспечить запуск всего окружения одной кнопкой, чтобы судьям не пришлось мучиться с настройкой.

Шаги:

1. docker-compose.yml (в корне):

- Настроить сервисы:
 - proof-server (Rust backend из Этапа 12 для SSS-3).
 - example-frontend (React приложение из Этапа 20, опционально, можно просто раздать статику).
 - *Опционально:* solana-test-validator (образ с solana-cli), который при старте делает деплой нашего контракта.

2. Написать в README: *"Для запуска полного стека (Backend + SDK + Frontend) используйте docker-compose up -d".*

»